Contents

1 Runtime-Inspectable Interfaces: Diagnostics as a First-Class UX Primitive **1**
1.1 Abstract . . . 1
1.2 1. Introduction . . . 1
1.3 Citation status and research gap . . . 2
1.4 2. Background and related work . . . 2
1.5 3. Core model . . . 3
1.6 4. Contributions . . . 3
1.7 5. Design implications . . . 3
1.8 6. Evaluation plan . . . 4
1.9 7. Implementation status and limitations . . . 4
1.10 8. Conclusion . . . 4

1 Runtime-Inspectable Interfaces: Diagnostics as a First-Class UX Primitive

> **Status: citation-complete expansion draft (editorial review copy).** Paper 13 of the Fundamental expansion family. This paper generalizes claims from the first eight Fundamental papers into a broader diagnostics paper. It is not yet externally peer reviewed, and any empirical section below is a study design rather than a result.

Author: Zach Shallbetter
Series: Fundamental Research Expansion Papers, Paper 13 of 30
Depends on: Paper 08, Paper 20
Primary source family: Papers 01–08 and the Fundamental Engine documentation [fundamental-site; fundamental-engine-repo]

1.1 Abstract

Interactive systems should expose inspectable explanations of their own behavior: not only source-level debugging for developers, but runtime-level answers to why an element changed, which relation acted, and what state caused the response. This paper isolates that claim from the implementation-specific language of the first Fundamental paper family and states it as a portable interface principle. The contribution is threefold. First, it defines the core concept in operational terms: what state must exist, where that state lives, who can read it, and what makes the claim falsifiable. Second, it positions the concept against adjacent interface traditions — diagnostics, devtools, explainability, observability, and end-user transparency — so the difference is not merely terminological. Third, it sketches a concrete evaluation and conformance plan that can be run without assuming that Fundamental is the only possible implementation. The paper is therefore a bridge: grounded in Fundamental's shipped vocabulary and documentation, but written so the model can be argued, criticized, and reproduced outside that codebase.

1.2 1. Introduction

Modern interfaces routinely contain more meaning than their rendered components expose. A result is not only a card; it has relevance, source support, uncertainty, recency, and relation to neighboring results. A document section is not only a block of text; it has position, prior attention, cross-references, and a role in the reader's mental map. A dashboard tile is not only a rectangle; it participates in a larger state of risk, priority, anomaly, and dependency. Conventional UI systems can render these properties, but they rarely make them first-class interface state.

The first Fundamental paper family approaches this problem through a relational field runtime. The flagship paper names the general model; the supporting papers test that model against reading, evidence, motion/accessibility, host portability, patterns, data binding, and diagnostics. This paper pulls one reusable claim out of that family: interactive systems should expose inspectable explanations of their own behavior: not only source-level debugging for developers, but runtime-level answers to why an element changed, which relation acted, and what state caused the response.

The point is not to argue that every interface should contain particles, simulated forces, or visual field lines. Those are representations. The deeper claim is about interface state. A system becomes interesting when relationships, evidence, attention, memory, diagnostics, accessibility equivalents, and data lifecycle stop being scattered across component props and one-off event handlers and become inspectable participants in a shared model.

This distinction matters because many expressive interfaces fail in the same pattern. They use motion, canvas, color, badges, animation, or layout to suggest a relation, but the relation itself is not available to the system. It cannot be inspected, tested, degraded for accessibility, serialized as a pattern, or bound to data in another view. Meaning appears at the surface but is not carried as a durable substrate. Fundamental's vocabulary is one answer to that failure, but the broader research question is whether interface systems should treat such meaning as a first-class layer.

1.3 Citation status and research gap

Verified source role. This paper's load-bearing background claims are keyed to: [chrome-devtools]; [firefox-devtools]; [lim2009]; [abdul2018]; [ragan2016]; [kulesza2013].

Remaining research gap. The remaining research gap is to measure whether runtime-inspectable interfaces reduce diagnosis time, author error, or misplaced trust compared with conventional devtools and documentation.

1.4 2. Background and related work

This paper's source package separates developer inspection from user-facing intelligibility. Browser DevTools documentation anchors the practical lineage of live, read-only inspection surfaces for running interfaces [chrome-devtools; firefox-devtools]. Intelligibility and explainable-systems work supports the claim that systems are more accountable when users or authors can ask why, why not, what if, and how behavior occurred [lim2009; abdul2018]. Provenance and explanatory-debugging work supports the narrower design requirement that explanations should expose the basis of a behavior and help authors correct or calibrate the system rather than merely display a static label [ragan2016; kulesza2013].

The adjacent systems differ by what they treat as primary. Component systems treat local component state as the organizing unit. Reactive renderers treat a view as a function of application state. Scene graphs treat drawable objects and their transform hierarchy as the durable model. Information visualization treats data structure and visual encoding as primary. Accessibility standards treat semantic availability and operability as non-negotiable constraints. Fundamental's field model intersects all of these but is not reducible to any one of them: it treats rendered elements, data records, relationships, metrics, accessibility equivalents, and diagnostics as participants in a shared interface substrate.

The distinction is important for this paper's scope. A semantic substrate is not merely a data model behind a view. It is also not merely a visualization of application state. It is the layer that determines what the interface can honestly say about itself: what exists, what relates, what changed, what should remain available when a visual channel disappears, and what can be inspected when behavior becomes confusing.

1.5 3. Core model

The paper's model can be stated in four terms.

Participant. A participant is anything that can emit, receive, store, transform, or explain interface state. Participants include DOM elements, data records, relationships, layout regions, user attention, events, diagnostics, and visual layers. A rendered object is a participant only if it can be addressed by the model; visibility alone is not participation.

Signal. A signal is a typed property that can move between participants or be read from them. Examples include attention, memory, confidence, contradiction, priority, risk, uncertainty, relationship strength, and diagnostic cause. Signals may be visualized, but they should not be confused with the visualization.

Binding. A binding connects a participant to a source of meaning. A data record binds to a body; a source binds to a claim; a citation binds to an evidence relation; a motion trail binds to a semantic state; a diagnostic view binds to the runtime state it reveals. The binding determines whether the representation can be checked.

Equivalence. An equivalence is the rule by which a signal survives when a channel changes. Motion can become static emphasis; color can become text or shape; a canvas rendering can become semantic DOM; a particle path can become a relationship label. Equivalence is what prevents expressive interfaces from becoming inaccessible when their preferred representation is unavailable.

The model is falsifiable. If an interface shows a relation but the relation cannot be queried, it is not a participant. If a signal appears visually but has no semantic source, it is a decoration, not semantic state. If a reduced or alternate rendering drops meaning that the primary rendering carried, equivalence has failed. If diagnostics mutate the state they claim to explain, the diagnostic claim is invalid.

1.6 4. Contributions

This paper contributes the following claims.

- Define runtime inspectability as a UX primitive distinct from source debugging.
- Classify diagnostic questions: cause, relation, state, cost, prediction, provenance.
- Explain the reveal-never-mutate discipline.
- Position field diagnostics against browser devtools and explainable systems.

Together these claims make the concept stronger than a metaphor. They state what must be implemented, what must be inspectable, and what can fail.

1.7 5. Design implications

The first implication is that interface behavior should be authored against explicit relations rather than against visual side effects. A designer should be able to name a support relation, a contradiction, a memory trace, an attention budget, or a priority field without smuggling that concept into an animation curve or color rule. This is the reason the pattern model matters: it separates product-language concepts from executable runtime tokens.

The second implication is that accessibility should be evaluated at the level of meaning. Turning off motion is not enough if motion was the only carrier of origin, direction, or causality. Removing color is not enough if color was the only carrier of confidence or risk. Replacing a canvas with fallback text is not enough if the relationships encoded by the canvas disappear. The substrate has to preserve the source meaning and expose an alternate carrier.

The third implication is that diagnostics should be part of the interface model rather than an afterthought. If the interface can compute a behavior, it should be able to expose why that behavior occurred. A runtime-

inspectable interface does not require every user to inspect it, but it gives authors, auditors, and advanced users a path to check the system's claims.

The fourth implication is that data binding should preserve relational meaning. Rendering an array of records into cards loses the graph unless links, confidence, uncertainty, and lifecycle are also bound. A relational interface should not flatten data and then rebuild its meaning with badges.

1.8 6. Evaluation plan

A mature version of this paper would be evaluated through a family of studies rather than one omnibus experiment.

The first study would compare a conventional interface against a substrate-aware version on tasks that require orientation, refinding, and relationship awareness. The expected outcomes would include time to recover context, accuracy of relation identification, and subjective orientation.

The second study would test trust calibration. Participants would answer source-backed questions using either citations and badges or an explicit evidence graph. Outcomes would include discrimination between supported and unsupported claims, calibration error, and over-reliance under time pressure.

The third study would test semantic degradation. Participants would use a full expressive surface and a degraded surface with motion, color, or canvas effects removed. A non-inferiority design would ask whether comprehension and orientation are preserved when the expressive channel disappears.

The fourth study would test authoring and debugging. Authors would diagnose why a behavior occurred with and without runtime diagnostics. Outcomes would include time-to-diagnose, fix accuracy, and perceived confidence.

The studies should be preregistered and analyzed with mixed-effects models where tasks and participants are crossed, following the methods pattern used in the first family [baayen2008; nosek2018].

1.9 7. Implementation status and limitations

This draft should not be read as a claim that all proposed behavior is currently shipped. The first family already distinguishes shipped runtime features, study-page demonstrations, proposed harnesses, and unrun evaluations. The same discipline applies here. Fundamental provides a concrete implementation vocabulary — bodies, fields, patterns, registries, diagnostics, data binding, host boundaries — but this expansion paper argues the broader model.

The main limitation is abstraction risk. A general paper can become too broad to falsify. To avoid that, every concept above is tied to a failure condition: unqueryable relations, unsupported signals, inaccessible channel loss, mutating diagnostics, and flattened data. Future revisions should preserve those failure tests.

A second limitation is empirical status. The external bibliography now supports the paper's background and evaluation frame, but the paper's central proposal still requires the study or implementation evidence named in the research gap.

1.10 8. Conclusion

Interactive systems should expose inspectable explanations of their own behavior: not only source-level debugging for developers, but runtime-level answers to why an element changed, which relation acted, and what state caused the response. The claim is broader than Fundamental but made concrete by it. The first paper family shows the pieces: reading memory, evidence graphs, reduced-motion equivalence, host-driven runtime structure, patterns, data binding, and diagnostics. This expansion paper turns those pieces into a

general research question: what would interface systems look like if relational meaning were treated as a substrate rather than an ornament?